LAPQ lib

Generated from `docs/lapq-lib.ipynb`

Abstract

This report describes the structure and current status of lapq, a C99 skip-list priority queue library with experimental learning-augmented insertion hints, instrumentation, benchmarks, Python bindings, and packaging support.

Contents

1 Introduction

`lapq` is a small experimental library for learning-augmented priority queues. The project combines a compact C99 core with a minimal Python binding, so that the data structure can be tested from C while also remaining usable from higher-level experimental code.

The central idea follows the learning-augmented priority queue model: predictions do not replace correct key comparisons, but they can provide hints that make the search for an insertion position faster. Correctness is always provided by the ordinary comparator, while hints may be perfect, noisy, stale, or invalid. The main theoretical reference is the work by Benomar and Coester on learning-augmented priority queues [BC24].

2 Repository Structure

The repository is organized around three main areas: the C library, the Python package, and the verification/documentation tooling.

- `include/lapq/lapq.h` exposes the public C API.
- `src/` contains the internal implementation, split into the public facade, skip-list logic, handle table, and statistics.
- `python/lapq/` contains the CPython extension and the Python module entry point.
- `tests/` contains deterministic C tests and Python tests.
- `benchmarks/` contains the C benchmark used to compare scenarios with and without hints.
- `docs/` contains this report, the LaTeX template used by `nbconvert`, the bibliography, and local reference papers.
- `.github/workflows/release.yml` describes the release pipeline for source distributions and platform wheels.

The `Makefile` is the main operational entry point. It builds the static C library, runs tests and sanitizer checks, launches the benchmark, builds the Python package, and regenerates this PDF with `make docs`.

3 C Core

The C core implements a generic priority queue: items are caller-owned `void *` pointers, and ordering is provided by a `lapq_cmp_fn` comparator. The public header keeps `struct lapq` opaque; implementation details live in `src/lapq_internal.h`.

The concrete data structure is a skip list with a fixed maximum level, `LAPQ_MAX_LEVEL`. Each node stores forward and backward pointers for every level, the user item, the randomly assigned height, and, when requested, a generational handle. The minimum is always the first node at level 0, so `peek_min` and `extract_min` operate directly on the head successor.

The source files are separated by responsibility:

- `src/lapq.c` handles queue lifetime and public API calls.
- `src/skiplist.c` handles random levels, predecessor search, linking, unlinking, and invariant checks.
- `src/handles.c` manages stable generational handles.
- `src/stats.c` centralizes comparison and traversal counters.

This split keeps the public interface small while isolating the experimental search logic inside the skip-list module.

4 Public API

The C API supports the standard priority queue operations: insertion, minimum inspection, minimum extraction, size queries, emptiness checks, and destruction. Operations that need to find an existing item again, such as `decrease_key` and arbitrary removal, use a `struct lapq_handle` returned at insertion time.

Handles contain the queue identifier, an internal slot, and a generation counter. When an item is removed, its slot can be recycled only after the generation has been incremented. This prevents stale handles from accidentally resolving to unrelated future items in the same slot and allows the implementation to reject handles from other queues or removed items.

Configuration is intentionally small. `struct lapq_config` provides a seed for deterministic skip-list level generation and a `flags` field. The main flag today is `LAPQ_ENABLE_STATS`, which enables experimental instrumentation.

5 Learning-Augmented Hints

`struct lapq_hint` is the API point where the library becomes learning-augmented. The supported hint kinds are:

- `LAPQ_HINT_NONE`, meaning no prediction;
- `LAPQ_HINT_PREDECESSOR`, a handle that is expected to point to the predecessor of the item being inserted;
- `LAPQ_HINT_RANK`, a predicted insertion rank.

Predecessor hints are the most developed case. If the handle is valid, the search starts at the hinted node instead of the head. The correction procedure handles both directions: if the hint is too far to the right, it moves backward; if the hint is too far to the left, it expands to the right and then refines the position top-down. If the hint is invalid, the library increments `invalid_hints` and falls back to an ordinary head search.

Rank hints are present as an experimental API, but they are currently converted into a starting node by walking level 0. They are useful for interface and correctness experiments, but they do not yet implement an indexed or vEB-like structure with stronger asymptotic guarantees.

6 Instrumentation

`struct lapq_stats` collects counters meant for experiments, not API-level complexity guarantees. The main counters measure:

- clean comparisons performed by the comparator;
- level-0 steps and express-level steps in the skip list;
- backward correction, bottom-up expansion, and top-down refinement steps;
- predecessor hints, rank hints, and invalid hints.

This instrumentation is mainly used by the benchmark to compare baseline insertions, perfect hints, noisy hints, and deliberately bad hints.

7 Python Binding

The Python package exposes a `PriorityQueue` class implemented in `python/lapq/_lapq.c`. The binding uses the C core and presents a small Python interface:

```
from lapq import PriorityQueue

queue = PriorityQueue()
queue.push(2.0, "b")
queue.push(1.0, "a")
assert queue.pop() == (1.0, "a")
```

Each Python item stores a `double` key, an insertion sequence number, and the associated Python value. The sequence number makes equal-priority items stable: for equal keys, the item inserted first is extracted first. The binding also exposes `peek`, `clear`, `empty`, `len(queue)`, `stats`, and `reset_stats`.

The Python binding does not yet expose handles, `decrease_key`, or predictive hints directly. Its current role is to provide a usable Python priority queue and to verify that the C core can be packaged as a CPython extension.

8 Tests and Benchmarks

The C test suite covers insertion/extraction order, handles and `decrease_key`, duplicate keys, good and bad hints, stale handles, hinted search phases, and a randomized oracle. Critical sequences call `lapq_check_invariants`, which verifies ordering, backward links, skip-list levels, queue size, and handle consistency.

The Python test suite verifies `push`/`pop` ordering, stability for equal priorities, `clear`, and statistics counters.

The C benchmark runs several scenarios:

- `baseline`: insertions without hints;
- `perfect`: perfect predecessor hints;
- `noisy`: predecessor hints with controlled error;
- `bad-left` and `bad-right`: deliberately distant hints;
- `decrease`: a scenario dedicated to `decrease_key`.

The output includes timings, clean comparisons, search steps, hint counts, invalid hints, average/maximum error, and a checksum. With `--csv`, the benchmark produces rows that can be imported into notebooks or analysis scripts.

9 Packaging and Release

The project is packaged with `setuptools`. `pyproject.toml` defines metadata, the project version, and the build backend; `setup.py` declares the `lapq._lapq` extension, compiling the Python binding and the C sources together. `MANIFEST.in` includes headers, sources, and Python files in the source distribution.

The GitHub Actions workflow in `.github/workflows/release.yml` is triggered by `v*` tags. It builds a source distribution and CPython wheels for Linux, macOS, and Windows using `cibuildwheel`, then runs a minimal import-and-use test against the generated wheels. The resulting artifacts are attached to a GitHub Release.

10 Status and Future Work

The library already implements a correct skip-list priority queue with experimental predecessor hints, rank hints, generational handles, instrumentation, benchmarks, and Python packaging. This makes it useful for controlled experiments on learning-augmented priority queue behavior.

Natural next steps include:

- exposing handles and hints through the Python API;
- implementing dirty comparisons, which are not present yet;
- replacing the current linear rank-hint handling with an indexed structure;
- adding Python experiments with plots and benchmark analysis;
- comparing empirical behavior against the theoretical guarantees of the learning-augmented model.

Bibliography

[BC24] Ziyad Benomar and Christian Coester. Learning-augmented priority queues. In *Advances in Neural Information Processing Systems 38*, 2024. Local copy: docs/papers/learning_augmented_priority_queues.pdf.